



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA EN SOFTWARE

DOCUMENTO DE ESPECIFICACIONES
NIVEL OPERATIVO - RETAILMIND SHOP

GA 07

Materia: Construcción de Software

Nombre: Benites Pérez Dariem Alberto

Sexto Semestre

Quevedo, Ecuador — 2026

Índice

Índice	2
1. Introducción.....	3
2. Niveles empresariales (resumen del nivel operativo).....	4
3. Especificación general del sistema operativo	5
4. Especificaciones por módulo	6
4.1 Autenticación y control de acceso	7
4.2 Catálogo de productos.....	11
4.3 Carrito de compras y checkout	15
4.4 Wishlist (lista de deseos).....	19
4.5 Pedidos del cliente (historial)	22
4.6 Perfil del cliente	25
4.7 Recomendaciones.....	28
5. Historias de usuario del nivel operativo	32
6. Matriz de trazabilidad consolidada	35
7. Hallazgos y deuda técnica.....	36

1. Introducción

RetailMind es una plataforma web de retail analytics con tienda online integrada que combina un pipeline ETL en Python, un motor de analítica sobre eventos de comportamiento y una experiencia de e-commerce completa, empaquetada en contenedores con Docker Compose. Su stack es Angular 17 (frontend SPA standalone), Spring Boot 3.5 (backend API con JWT) y ClickHouse (esquema estrella) como única base de datos operativa; PostgreSQL fue eliminado y su DDL queda como legacy. El sistema se organiza en tres niveles empresariales —operativo (genera ventas), táctico (decisiones) y estratégico (KPIs ejecutivos)—; este documento cubre exclusivamente el nivel operativo.

2. Niveles empresariales (resumen del nivel operativo)

La arquitectura por niveles (Principio III de la constitución) organiza el dominio en operativo, táctico y estratégico. Esta especificación cubre el nivel operativo. La siguiente tabla resume, por módulo operativo, sus objetivos estratégicos (OE), tácticos (OT) y operativos (OO) y los casos de uso (CU) asociados.

Módulo operativo	OE	OT	OO	CU
Autenticación	OE-04	OT-07, OT-08	OO-12, OO-14	CU-01, CU-02
Catálogo	OE-01	OT-01	OO-01	CU-04, CU-05, CU-06
Carrito y checkout	OE-01	OT-01	OO-02	CU-07, CU-08, CU-09
Wishlist	OE-01	OT-02	OO-03	CU-10, CU-11, CU-12
Pedidos	OE-01	OT-02	OO-04	CU-13, CU-14
Perfil	OE-04	OT-07	— (sin OO en EVF04)	CU-008 (TA06)
Recomendaciones	OE-01, OE-04	OT-02	— (sin OO en EVF04)	CU-009 (TA06)

Leyenda de objetivos:

ID	Descripción
OE-01	Maximizar conversiones y ventas en la tienda online.
OE-04	Garantizar la seguridad (RBAC) e inteligencia de negocio / personalización.
OT-01	Mejorar la experiencia de navegación del cliente.
OT-02	Incrementar el engagement y la retención del cliente.
OT-07	Gestionar de forma segura identidades y credenciales.
OT-08	Separar y proteger vistas según rol.
OO-01	Facilitar la búsqueda y filtrado de productos por categoría, marca y precio.
OO-02	Agilizar el proceso de compra desde la selección hasta el checkout.
OO-03	Permitir guardar productos de interés en una lista de deseos.
OO-04	Dar seguimiento completo a los pedidos del cliente.
OO-12	Autenticar con JWT y credenciales hasheadas en ClickHouse.
OO-14	Mostrar interfaces según rol.

3. Especificación general del sistema operativo

Objetivo general

Proveer la capa que genera ventas y engagement: permitir que un visitante explore el catálogo público, que un cliente autenticado compre (carrito y checkout), guarde productos (wishlist), consulte su historial de pedidos, gestione su perfil y reciba recomendaciones personalizadas, todo sobre una base segura de autenticación JWT y control de acceso por roles (RBAC).

Actores

- **Cliente:** usuario autenticado de la tienda; tras login va a /shop.
- **Administrador (ADMIN):** tras login va a /dashboard; único rol que registra usuarios y accede a analítica avanzada, ETL, gestión y reportes.
- **Motor de recomendaciones:** componente lógico que calcula el scoring, selecciona productos y aplica el fallback.
- **Sistema ETL:** pipeline Python que extrae el dataset de PocketBase, lo materializa en Parquet y lo carga en ClickHouse.

Módulos operativos

- 001 Autenticación y control de acceso
- 002 Catálogo de productos
- 003 Carrito de compras y checkout
- 004 Wishlist (lista de deseos)
- 005 Pedidos del cliente (historial)
- 006 Perfil del cliente
- 007 Recomendaciones

Reglas y restricciones generales (constitución)

- Plataforma web stateless (JWT STATELESS) para escalado horizontal; cada componente con su propio Dockerfile.
- El negocio vende primero: los flujos operativos tienen prioridad sobre analítica y reportes.
- ClickHouse como única fuente operativa, con JdbcTemplate y consultas parametrizadas (sin JPA); PostgreSQL es legacy.
- Seguridad por defecto (no negociable): JWT con Spring Security STATELESS, BCrypt y RBAC (ADMIN, CLIENTE). Catálogo público; tienda y datos de usuario requieren autenticación; analítica/ETL/gestión/reportes solo ADMIN.
- Acceso por defecto: denegar lo no declarado. Los secretos provienen de variables de entorno y no se versionan.
- Verificación: todo cambio debe compilar/construir (mvn clean install, npm run build, ETL) y cubrir con pruebas las rutas críticas de venta y seguridad.

4. Especificaciones por módulo

Cada módulo sigue la misma plantilla de 14 apartados (estilo SDD). Los KPI por módulo, la trazabilidad por módulo y los pendientes se consolidan una sola vez al final del documento (secciones 6 y 7).

4.1 Autenticación y control de acceso

Feature Branch: 001-operativo-autenticacion

1. Nombre de la funcionalidad

Autenticación y control de acceso

2. Objetivo

Autenticar al usuario con username y password validados contra usuarios_sistema (ClickHouse), emitir un JWT con el rol embebido, mantener la sesión mediante refresh token y controlar el acceso a vistas y endpoints según el rol (RBAC con ADMIN y CLIENTE). Es la base de seguridad sobre la que operan los demás módulos.

3. Usuarios / actores

- **Visitante no autenticado:** solo accede a recursos públicos (catálogo, login).
- **CLIENTE:** usuario autenticado; tras login va a /shop.
- **ADMIN:** usuario autenticado; va a /dashboard y es el único que registra usuarios.
- **Sistema (backend):** emite y valida tokens, aplica el filtro de seguridad y siembra el admin inicial.

El enum de roles define ADMIN, CLIENTE y VIEWER; VIEWER existe pero no se usa en ningún flujo (PENDIENTE).

4. Contexto del problema

Sin control de acceso, RetailMind expone datos de negocio y operaciones de venta a usuarios no autorizados. Se necesita autenticación sin estado (apta para escalado horizontal) y autorización por rol que separe la experiencia de cliente de la de administración, sin revelar en los errores si falló el usuario o la contraseña, manteniendo la sesión mediante refresh token.

5. Requisitos funcionales

ID	Requisito
RF-AUT-001	Login vía POST /api/auth/login (username, password).
RF-AUT-002	Validar credenciales contra retailmind.usuarios_sistema (ClickHouse, no PostgreSQL).
RF-AUT-003	Verificar la contraseña contra el hash BCrypt almacenado.
RF-AUT-004	Rechazar el login de usuarios inactivos (activo = 0).
RF-AUT-005	Emitir token JWT de acceso con subject = username y claim rol.
RF-AUT-006	Emitir un refresh token JWT independiente del de acceso.
RF-AUT-007	Respuesta de login: token, refreshToken, username, nombre, rol, expiresIn.
RF-AUT-008	Renovar el acceso vía POST /api/auth/refresh validando el refresh vigente.
RF-AUT-009	Exponer la identidad vía GET /api/auth/me (username, rol).
RF-AUT-010	POST /api/auth/logout; el cierre efectivo lo hace el cliente borrando token, refresh y datos.
RF-AUT-011	Alta de usuarios vía POST /api/auth/register, solo ADMIN (valida rol y unicidad; guarda hash BCrypt).
RF-AUT-012	El cliente persiste la sesión en localStorage (token, refresh, datos).
RF-AUT-013	El cliente adjunta Authorization: Bearer <token>, salvo login, refresh y health.
RF-AUT-014	Ante un 401, un único intento de refresh automático; si falla, cierra sesión.
RF-AUT-015	Reintento de GET ante error de red (status 0) o ≥ 500 : hasta 2 veces con 1000 ms de espera.

ID	Requisito
RF-AUT-016	Proteger rutas con authGuard (sesión) y adminGuard (rol ADMIN).
RF-AUT-017	Tras login: ADMIN → /dashboard; cualquier otro rol → /shop.
RF-AUT-018	Validación en cliente antes de enviar: username ≥ 3 y password ≥ 6 caracteres.
RF-AUT-019	Filtro JWT valida el token en cada petición; token ausente o inválido no autentica.
RF-AUT-020	Al arrancar, sembrar un usuario admin (rol ADMIN) si no existe.

6. Requisitos no funcionales

ID	Requisito
RNF-AUT-001	El token de acceso expira a las 24 h (86 400 000 ms).
RNF-AUT-002	El refresh token expira a los 7 días (604 800 000 ms).
RNF-AUT-003	JWT firmado con HMAC-SHA y clave ≥ 256 bits (se rellena a 32 bytes si es menor).
RNF-AUT-004	Contraseñas con BCrypt (coste 10); nunca en texto plano.
RNF-AUT-005	Backend sin estado (STATELESS) para habilitar escalado horizontal.
RNF-AUT-006	CORS solo desde http://localhost:4200 (GET/POST/PUT/DELETE/OPTIONS, con credenciales).
RNF-AUT-007	Fallo de login → HTTP 401 con mensaje genérico que no revela la causa.
RNF-AUT-008	El login consulta las credenciales de forma parametrizada.
RNF-AUT-009	Registro de eventos en logs/retailmind.log (nivel INFO).
RNF-AUT-010	PENDIENTE: no existe un umbral de latencia medido para el login.

7. Reglas de negocio

ID	Regla
RN-AUT-001	Solo los usuarios con activo = 1 pueden autenticarse.
RN-AUT-002	El mensaje ante credenciales inválidas es genérico y no distingue la causa.
RN-AUT-003	El rol define la vista inicial: ADMIN → /dashboard; otros → /shop.
RN-AUT-004	Roles aceptados en el registro: ADMIN y CLIENTE.
RN-AUT-005	El username debe ser único.
RN-AUT-006	El alta de usuarios solo la ejecuta un ADMIN; no hay auto-registro público.
RN-AUT-007	El claim rol del token es la fuente de verdad para la autorización.
RN-AUT-008	La renovación emite un nuevo token de acceso y conserva el refresh vigente.

8. Entradas

- Login (POST /api/auth/login): { username, password }.
- Refresh (POST /api/auth/refresh): { refreshToken }.
- Identidad / Logout (GET /me, POST /logout): cabecera Authorization: Bearer <token>.
- Registro (POST /api/auth/register, solo ADMIN): { username, password, rol: ADMIN|CLIENTE, email? }. El email se guarda hoy en el campo nombre.
- Formulario de login: username ≥ 3 y password ≥ 6 caracteres.

9. Salidas

- **Login 200:** { token, refreshToken, username, nombre, rol, expiresIn }.
- **Refresh 200:** token nuevo y mismo refreshToken.
- **/me 200:** { username, rol }. Logout 200 y Register 200 { success, mensaje }.
- **Errores:** Login 401 "Credenciales incorrectas"; Refresh 401; Register 400 (faltan campos / duplicado / rol inválido); Register 500.
- **UI:** "Credenciales incorrectas. Intenta de nuevo." + sacudida; snackbar ante 500 o sin conexión (status 0).

10. Escenarios (Gherkin)

Escenario 1 — Login exitoso (CU-01)

Dado un usuario activo con credenciales válidas en usuarios_sistema
 Cuando envía POST /api/auth/login con datos correctos
 Entonces responde 200 con token, refreshToken y datos, y el JWT incluye el claim rol

Escenario 2 — Login fallido

Dado un intento de inicio de sesión
 Cuando el usuario es inexistente, la password es incorrecta o la cuenta está inactiva
 Entonces responde 401 "Credenciales incorrectas" sin revelar la causa

Escenario 3 — Registro exitoso (CU-02, solo ADMIN)

Dado un ADMIN autenticado
 Cuando envía POST /api/auth/register con username nuevo, password y rol válido
 Entonces responde 200, la contraseña se guarda en BCrypt y el usuario queda activo

Escenario 4 — Renovación de sesión

Dado una petición protegida que recibe 401
 Cuando el cliente intenta renovar con el refresh token
 Entonces obtiene un nuevo token de acceso; si la renovación falla, cierra la sesión

11. Criterios de aceptación

ID	Criterio
CA-AUT-001	Credenciales correctas y cuenta activa → 200 con JWT cuyo claim rol coincide.
CA-AUT-002	Usuario inexistente, password incorrecta o cuenta inactiva → 401 "Credenciales incorrectas".
CA-AUT-003	Las contraseñas nunca se almacenan ni transmiten en texto plano (siempre BCrypt).
CA-AUT-004	El token deja de aceptarse a las 24 h y el refresh a los 7 días.
CA-AUT-005	Tras login, ADMIN aterriza en /dashboard y CLIENTE en /shop.
CA-AUT-006	Sin sesión a ruta protegida → /login; no-ADMIN a ruta admin → /dashboard.
CA-AUT-007	Un 401 dispara un único refresh; si falla, se cierra la sesión.
CA-AUT-008	register sin rol ADMIN es rechazado (no crea usuario).
CA-AUT-009	register rechaza username duplicado, rol no permitido y campos faltantes con 400.
CA-AUT-010	GET /me con token válido devuelve username y rol; sin token, 401.

12. Restricciones

- Base de datos única: credenciales solo en retailmind.usuarios_sistema (ClickHouse); PostgreSQL eliminado.
- Acceso a datos con JdbcTemplate (sin JPA); login parametrizado.
- Stack fijado: Spring Boot 3.5 + Spring Security + jjwt 0.12.3 / Angular 17 standalone (guards e interceptor funcionales).

- Sin estado en servidor: la sesión vive en localStorage + JWT.
- CORS limitado al origen del frontend (http://localhost:4200).

13. Dependencias

- ClickHouse con la base retailmind y la tabla usuarios_sistema (se crea al arrancar si no existe).
- Variables de entorno: JWT_SECRET, JWT_EXPIRATION, JWT_REFRESH_EXPIRATION y datasource de ClickHouse.
- Spring Security (AuthenticationManager, BCryptPasswordEncoder, filtro JWT).
- Frontend: AuthService, authGuard, adminGuard, authInterceptor, MatSnackBar.
- Constitución del proyecto, Principio V (seguridad por defecto: JWT + RBAC).

14. Fuera de alcance

- Administración de usuarios por ADMIN (OO-13): módulo administrativo aparte.
- Gestión de perfil y cambio de contraseña por el propio usuario (módulo Perfil).
- Auto-registro público de clientes y recuperación de contraseña: no implementados.
- MFA, SSO/OAuth: no implementados.
- Bloqueo de cuenta / rate limiting por intentos fallidos: no implementado.
- Invalidación de tokens en servidor (logout server-side / blacklist): no implementada.
- Rol VIEWER: definido pero sin flujos asociados.

4.2 Catálogo de productos

Feature Branch: 002-operativo-catalogo

1. Nombre de la funcionalidad

Catálogo de productos

2. Objetivo

Permitir que cualquier visitante (autenticado o no) explore el catálogo de RetailMind, lo filtre por categoría, marca y rango de precio, y consulte el detalle de un producto. Cada vista de detalle registra un evento view en fact_eventos. Es la puerta de entrada a la venta: maximiza la exposición de productos para favorecer la conversión.

3. Usuarios / actores

- **Visitante no autenticado:** lista, filtra y ve el detalle (catálogo público).
- **CLIENTE autenticado:** además puede agregar al carrito o wishlist (otros módulos) y aporta su user_id real a los eventos.
- **Sistema (backend):** sirve productos, categorías y marcas desde ClickHouse y persiste eventos en fact_eventos.

4. Contexto del problema

RetailMind necesita exponer su inventario (~1.200 productos) de forma navegable y filtrable para que el cliente encuentre rápido lo que busca; una navegación pobre reduce la conversión. Además, el negocio necesita capturar las vistas para alimentar la analítica. Por eso el catálogo es público y registra eventos de comportamiento sin interrumpir la navegación.

5. Requisitos funcionales

ID	Requisito
RF-CAT-001	Listar productos paginados vía GET /api/catalogo/productos (endpoint público).
RF-CAT-002	Incluir solo productos activos (activo = 1).
RF-CAT-003	Filtrar por categoría (categoria_id).
RF-CAT-004	Filtrar por marca (brand).
RF-CAT-005	Filtrar por rango de precio (min_price, max_price).
RF-CAT-006	Paginación con page (def 0) y size (def 20): content, totalElements, totalPages, number, size.
RF-CAT-007	Cada producto: productold, nombre, descripcion, categoriald, categoriaNombre (JOIN dim_categoria), brand, price, stock, imagenUrl.
RF-CAT-008	Detalle vía GET /api/catalogo/productos/{id}; 404 si no existe.
RF-CAT-009	Listar categorías con conteo de productos activos (GET /api/catalogo/categorias).
RF-CAT-010	Listar marcas distintas de productos activos (GET /api/catalogo/marcas).
RF-CAT-011	Registrar eventos vía POST /api/catalogo/eventos en fact_eventos.
RF-CAT-012	Al abrir el detalle, registrar automáticamente un evento user_action = "view".
RF-CAT-013	Si no hay session_id, generar uno con formato sess_shop_<8 hex>.
RF-CAT-014	Derivar is_conversion = 1 si purchase, drop_off_flag = 1 si drop, y la semana ISO actual.
RF-CAT-015	La tienda muestra un grid con filtro por categoría (toggle) y paginador.
RF-CAT-016	Navegar al detalle desde la tarjeta (/shop/producto/{id}).

ID	Requisito
RF-CAT-017	Los endpoints de catálogo son accesibles sin autenticación.

6. Requisitos no funcionales

ID	Requisito
RNF-CAT-001	Tamaño de página por defecto 20 en backend; la tienda usa 12.
RNF-CAT-002	Paginación base 0.
RNF-CAT-003	Endpoints de catálogo sin token JWT (acceso público).
RNF-CAT-004	Listado ordenado de forma estable por producto_id.
RNF-CAT-005	Un fallo al registrar evento no interrumpe la navegación (se registra en log).
RNF-CAT-006	Acceso a datos con JdbcTemplate sobre ClickHouse (sin JPA).
RNF-CAT-007	Ante error en la consulta, degradar a conjunto vacío en vez de exponer el error.
RNF-CAT-008	PENDIENTE: no existe umbral de latencia medido para listado o detalle.

7. Reglas de negocio

ID	Regla
RN-CAT-001	Solo se muestran productos con activo = 1.
RN-CAT-002	Categorías y marcas se derivan solo de productos activos.
RN-CAT-003	El evento view se registra en el detalle, no en el grid.
RN-CAT-004	Sin usuario, el evento usa user_id = "anonymous" y canal "web".
RN-CAT-005	is_conversion = 1 solo si purchase; drop_off_flag = 1 solo si drop.
RN-CAT-006	El catálogo es público: no requiere sesión para listar, filtrar ni ver detalle.
RN-CAT-007	La selección de categoría es excluyente y tipo toggle.
RN-CAT-008	Las 8 categorías: Electronics, Groceries, Sports, Accessories, Beauty, Home, Shoes, Apparel.

8. Entradas

- **Listar (GET /productos):** categoria_id, brand, min_price, max_price, page (def 0), size (def 20) — todos opcionales.
- **Detalle (GET /productos/{id}):** productid en la ruta.
- **Categorías / Marcas:** sin parámetros.
- **Evento (POST /eventos):** { user_id?, product_id?, user_action, channel?, price?, session_id? }.

9. Salidas

- **Listar 200:** { content:[...], totalElements, totalPages, number, size }.
- **Detalle 200:** objeto producto; 404 si no existe.
- **Categorías 200:** [{ categoryid, nombre, total }]. **Marcas 200:** ["Marca1", ...]. **Evento 200:** { success: true }.
- **Degradación:** ante fallo interno del listado devuelve estructura vacía con HTTP 200 (oculta el error al cliente).

10. Escenarios (Gherkin)

Escenario 1 — Ver catálogo (CU-04)

Dado un visitante en la tienda
 Cuando solicita GET /api/catalogo/productos

Entonces recibe 200 con productos activos paginados (sin token)

Escenario 2 — Filtrar (CU-05)

Dado el catálogo cargado

Cuando aplica categoria_id, brand o min_price/max_price

Entonces el resultado se acota a los filtros indicados

Escenario 3 — Detalle con evento "view" (CU-05)

Dado un producto existente

Cuando el usuario abre su detalle (GET /productos/{id})

Entonces se devuelve el producto y se registra un evento view en fact_eventos

Escenario 4 — Sin resultados

Dado un filtro sin coincidencias

Cuando se solicita el listado

Entonces responde content: [] y totalElements: 0 con HTTP 200

11. Criterios de aceptación

ID	Criterio
CA-CAT-001	GET /productos responde 200 sin token y solo con productos activos.
CA-CAT-002	Los filtros categoria_id, brand y min/max_price acotan correctamente el resultado.
CA-CAT-003	El listado trae content, totalElements, totalPages, number y size (def 20, base 0).
CA-CAT-004	Cada producto incluye categoriaNombre resuelto vía JOIN con dim_categoria.
CA-CAT-005	GET /productos/{id} devuelve el detalle si existe y 404 si no.
CA-CAT-006	Al abrir el detalle se registra un evento view con channel "web" y user_id real o "anonymous".
CA-CAT-007	GET /categorias trae el total por categoría; GET /marcas trae marcas distintas.
CA-CAT-008	Un fallo al registrar el evento no interrumpe el detalle.
CA-CAT-009	Filtros sin coincidencias → content: [] y totalElements: 0 con HTTP 200.

12. Restricciones

- Datos en retailmind.productos_catálogo y fact_eventos; categorías contra dim_categoria.
- Acceso a datos con JdbcTemplate (sin JPA).
- Endpoints públicos (definido en SecurityConfig).
- Persistencia de eventos best-effort: prioriza no romper la navegación sobre garantizar la inserción.
- Stack fijado: Spring Boot 3.5 / Angular 17 standalone con Angular Material.

13. Dependencias

- ClickHouse con productos_catálogo, dim_categoria y fact_eventos pobladas (etl/extraccion/13_create_shop_tables.py).
- Módulo de Autenticación (001): provee el user_id real cuando hay sesión.
- Carrito (003), Wishlist (004) y Recomendaciones (007): se consumen desde la tarjeta/detalle pero son módulos aparte.
- Constitución, Principios II (operativa primero), III (niveles) y IV (ClickHouse + datos).

14. Fuera de alcance

- Agregar al carrito y wishlist: módulos independientes (solo se relacionan desde la tarjeta/detalle).

- Recomendaciones / productos similares: módulo independiente consumido desde el detalle.
- Búsqueda por texto libre: el campo existe en la UI pero no está cableado (PENDIENTE); el backend no la implementa.
- Filtros de marca y precio en la UI del grid: soportados por la API pero no cableados (PENDIENTE).
- CRUD de productos del catálogo: corresponde al módulo administrativo de gestión de datos.

4.3 Carrito de compras y checkout

Feature Branch: 003-operativo-carrito

1. Nombre de la funcionalidad

Carrito de compras y checkout

2. Objetivo

Permitir que un cliente autenticado agregue productos al carrito, revise su contenido, elimine ítems y finalice la compra (checkout). El checkout genera una orden en ordenes/orden_items, registra un evento purchase por producto en fact_eventos (conversión) y vacía el carrito. Es el módulo que materializa la venta: convierte la navegación en órdenes.

3. Usuarios / actores

- **CLIENTE autenticado:** agrega, consulta, elimina ítems y finaliza la compra. Actor principal.
- **ADMIN autenticado:** puede operar el carrito como cualquier usuario autenticado.
- **Sistema (backend):** persiste el carrito, congela el precio al agregar, genera la orden y registra eventos.

*El módulo requiere autenticación: las rutas /api/carrito/** caen bajo authenticated() y /shop/carrito está protegida por authGuard. Todo opera sobre el username en sesión.*

4. Contexto del problema

La navegación solo genera valor si el cliente concreta la compra. RetailMind necesita un carrito persistente por usuario y un checkout que cree la orden y deje traza del embudo (add_to_cart → purchase). Sin este módulo el negocio no vende. Además debe impedir compras inválidas (carrito vacío) y capturar los eventos que permiten medir la conversión.

5. Requisitos funcionales

ID	Requisito
RF-CAR-001	Agregar producto vía POST /api/carrito/agregar (requiere autenticación).
RF-CAR-002	Al agregar, obtener el precio actual del producto; si no existe, error 400.
RF-CAR-003	Persistir en carrito_items: carrito_id (UUID), user_id, producto_id, cantidad, precio_unitario (congelado), fecha_agregado, activo = 1.
RF-CAR-004	Al agregar, registrar evento add_to_cart en fact_eventos.
RF-CAR-005	Listar ítems activos vía GET /api/carrito/{userId} (JOIN con productos_catalogo, orden fecha desc).
RF-CAR-006	Eliminar ítem vía DELETE /api/carrito/{userId}/{productid} con borrado lógico (activo = 0).
RF-CAR-007	Al eliminar, registrar evento drop en fact_eventos.
RF-CAR-008	El cliente calcula y muestra el total (suma de precioUnitario × cantidad).
RF-CAR-009	Finalizar compra vía POST /api/carrito/{userId}/checkout.
RF-CAR-010	Rechazar checkout con carrito vacío → 400 "El carrito esta vacio".
RF-CAR-011	Generar ordenId con formato ORD-<8 hex en mayúsculas>.
RF-CAR-012	Insertar cada ítem en orden_items (orden_id, producto_id, cantidad, precio_unitario).
RF-CAR-013	Registrar un evento purchase (conversión) por cada producto.
RF-CAR-014	Crear la orden en ordenes con total, estado "COMPLETADA", canal "web" y fecha_orden.
RF-CAR-015	Tras el checkout, vaciar el carrito (activo = 0 en todos los ítems).

ID	Requisito
RF-CAR-016	Devolver { ordenId, total, items } y mostrar la confirmación con el ordenId.
RF-CAR-017	Todas las operaciones requieren usuario autenticado.

6. Requisitos no funcionales

ID	Requisito
RNF-CAR-001	Endpoints /api/carrito/** requieren token JWT válido.
RNF-CAR-002	Cantidad por defecto al agregar = 1.
RNF-CAR-003	El precio_unitario se congela al agregar (snapshot), independiente del catálogo.
RNF-CAR-004	Borrado lógico y vaciado de forma síncrona (mutations_sync = 1).
RNF-CAR-005	Toda orden del checkout nace con estado "COMPLETADA" y canal "web".
RNF-CAR-006	Acceso a datos con JdbcTemplate sobre ClickHouse (sin JPA).
RNF-CAR-007	PENDIENTE: el checkout ejecuta varias mutaciones sin transacción (ClickHouse no es transaccional).
RNF-CAR-008	PENDIENTE: no existe umbral de latencia medido para agregar ni checkout.

7. Reglas de negocio

ID	Regla
RN-CAR-001	No se permite checkout con carrito vacío.
RN-CAR-002	Solo los ítems con activo = 1 cuentan para el carrito y el checkout.
RN-CAR-003	El precio del ítem es el del momento de agregarlo (snapshot).
RN-CAR-004	Toda orden por checkout queda COMPLETADA; no hay estados intermedios ni pago.
RN-CAR-005	Cada acción del embudo es un evento: add_to_cart, drop y purchase (is_conversion = 1).
RN-CAR-006	Eliminar un producto desactiva todas sus filas activas del usuario.
RN-CAR-007	El módulo opera sobre el username en sesión.

8. Entradas

- **Agregar (POST /agregar):** { user_id, producto_id, cantidad? (def 1) }.
- **Listar (GET /{userId}) / Eliminar (DELETE /{userId}/{productid}):** parámetros en la ruta.
- **Checkout (POST /{userId}/checkout):** userId en la ruta, cuerpo vacío.
- **Autenticación:** Authorization: Bearer <token> en todas las operaciones.

9. Salidas

- **Agregar 200 / Eliminar 200:** { success: true, mensaje }.
- **Listar 200:** [{ carritoId, productid, cantidad, precioUnitario, fechaAgregado, nombre, brand, categoriald }].
- **Checkout 200:** { ordenId: "ORD-XXXXXXX", total, items }.
- **Errores:** Agregar 400 (producto no encontrado); Checkout 400 "El carrito esta vacio"; 500 en operaciones.
- **UI:** "Compra realizada con exito!" con el ordenId tras el checkout.

10. Escenarios (Gherkin)

Escenario 1 — Agregar (CU-07)

Dado un cliente autenticado viendo un producto existente

Cuando envía POST /api/carrito/agregar

Entonces se congela el precio, se persiste el ítem (activo = 1) y se registra add_to_cart

Escenario 2 — Eliminar (CU-08)

Dado un cliente con productos en el carrito

Cuando envía DELETE /api/carrito/{userId}/{productid}

Entonces las filas del producto pasan a activo = 0 y se registra un evento drop

Escenario 3 — Checkout exitoso (CU-09)

Dado un carrito con al menos un ítem activo

Cuando envía POST /api/carrito/{userId}/checkout

Entonces genera ORD-XXXXXXX, inserta orden_items, registra purchase por producto, crea la orden COMPLETADA (canal web) y vacía el carrito

Escenario 4 — Checkout con carrito vacío

Dado un carrito sin ítems activos

Cuando envía el checkout

Entonces responde 400 "El carrito esta vacio" sin crear orden ni eventos

11. Criterios de aceptación

ID	Criterio
CA-CAR-001	Agregar un producto existente crea fila activa con precio congelado y genera add_to_cart.
CA-CAR-002	Agregar un producto inexistente → 400 sin crear fila.
CA-CAR-003	GET /{userId} devuelve solo ítems activos con nombre, marca y categoría (orden fecha desc).
CA-CAR-004	Eliminar marca activo = 0, genera drop y deja de aparecer.
CA-CAR-005	El total equivale a la suma de precioUnitario × cantidad de los ítems activos.
CA-CAR-006	Checkout no vacío crea la orden, inserta orden_items, registra purchase por producto y vacía el carrito.
CA-CAR-007	El checkout devuelve ordenId ORD- + 8 hex en mayúsculas, total e ítems.
CA-CAR-008	Checkout con carrito vacío → 400 "El carrito esta vacio" sin crear orden.
CA-CAR-009	Todas las operaciones rechazan peticiones sin token JWT válido.

12. Restricciones

- Carrito en retailmind.carrito_items; órdenes en ordenes/orden_items; eventos en fact_eventos.
- JdbcTemplate (sin JPA); getCarrito parametrizado, el resto por concatenación (PENDIENTE de seguridad).
- Autenticación obligatoria (no opera de forma anónima).
- Sin transacciones: el checkout no es atómico. Sin pago real: la orden nace COMPLETADA.
- Stack fijado: Spring Boot 3.5 / Angular 17 standalone con Angular Material.

13. Dependencias

- Módulo de Catálogo (002): obtiene el precio al agregar y reutiliza registrarEvento.
- Módulo de Autenticación (001): provee el username en sesión.
- Módulo de Pedidos (005): consume las tablas ordenes/orden_items que este módulo genera.
- ClickHouse con carrito_items, ordenes, orden_items, productos_catálogo y fact_eventos.
- Constitución, Principios II (operativa primero) y V (seguridad).

14. Fuera de alcance

- Historial de pedidos y detalle de orden (00-04): módulo independiente que lee estas órdenes.

- Wishlist: módulo independiente.
- Modificar la cantidad de una línea existente: no implementado (cada agregar inserta una fila nueva).
- Pasarela de pago y estados de orden: no implementados; la orden nace COMPLETADA.
- Validación/descuento de stock: no implementado (no se verifica al agregar ni en checkout).
- Consolidación de líneas duplicadas del mismo producto: no implementada.

4.4 Wishlist (lista de deseos)

Feature Branch: 004-operativo-wishlist

1. Nombre de la funcionalidad

Wishlist (lista de deseos)

2. Objetivo

Permitir que un cliente autenticado guarde productos de interés en una lista de deseos, la consulte y elimine productos, con un control visual de corazón (toggle) en el catálogo y el detalle. Cada alta registra un evento wishlist en fact_eventos. Busca incrementar la retención y el retorno del cliente.

3. Usuarios / actores

- **CLIENTE autenticado:** agrega, consulta, elimina y puede mover un producto al carrito. Actor principal.
- **ADMIN autenticado:** opera la wishlist como cualquier usuario autenticado.
- **Sistema (backend):** persiste en wishlist_items, evita duplicados y registra el evento wishlist al agregar.

Requiere autenticación: rutas /api/wishlist/** bajo authenticated() y vistas (/wishlist, /shop) protegidas por authGuard. Opera sobre el username en sesión.

4. Contexto del problema

La conversión inmediata no siempre ocurre: el cliente descubre productos que le interesan pero no compra en el momento. Sin lista de deseos, ese interés se pierde. RetailMind necesita capturar el engagement, facilitar el retorno del cliente y dejar traza de qué productos despiertan deseo. La wishlist es personal, evita duplicados y permite mover un producto al carrito.

5. Requisitos funcionales

ID	Requisito
RF-WIS-001	Agregar a la wishlist vía POST /api/wishlist/agregar (requiere autenticación).
RF-WIS-002	Rechazar duplicado → 400 "El producto ya esta en tu wishlist".
RF-WIS-003	Persistir en wishlist_items: wishlist_id (UUID), user_id, producto_id, fecha_agregado.
RF-WIS-004	Al agregar, registrar evento wishlist en fact_eventos.
RF-WIS-005	Listar vía GET /api/wishlist/{userId} (JOIN con productos_catálogo, orden fecha desc).
RF-WIS-006	Eliminar vía DELETE /api/wishlist/{userId}/{productid} (borrado físico).
RF-WIS-007	Control de corazón (toggle) en grid y detalle: agrega o elimina según el estado.
RF-WIS-008	La página de wishlist permite eliminar, mover al carrito y navegar al detalle.
RF-WIS-009	Todas las operaciones requieren usuario autenticado.

6. Requisitos no funcionales

ID	Requisito
RNF-WIS-001	Endpoints /api/wishlist/** requieren token JWT válido.
RNF-WIS-002	Unicidad por par (user_id, producto_id): no se admite el mismo producto dos veces.
RNF-WIS-003	Eliminación síncrona (mutations_sync = 1) por borrado físico.
RNF-WIS-004	Listado ordenado por fecha_agregado descendente.

ID	Requisito
RNF-WIS-005	Acceso a datos con JdbcTemplate sobre ClickHouse (sin JPA).
RNF-WIS-006	PENDIENTE: no existe umbral de latencia medido para agregar, listar o eliminar.

7. Reglas de negocio

ID	Regla
RN-WIS-001	Un producto no puede estar más de una vez en la wishlist de un usuario.
RN-WIS-002	Solo la acción de agregar registra evento; la eliminación no registra evento.
RN-WIS-003	El módulo opera sobre el username en sesión.
RN-WIS-004	La wishlist no afecta stock ni precio: es una lista de interés.
RN-WIS-005	Al agregar no se valida que el producto exista (a diferencia del carrito).
RN-WIS-006	El corazón refleja pertenencia: lleno si está en la wishlist, vacío si no.

8. Entradas

- **Agregar (POST /agregar):** { user_id, producto_id }.
- **Listar (GET /{userId}) / Eliminar (DELETE /{userId}/{productid}):** parámetros en la ruta.
- **Autenticación:** Authorization: Bearer <token>.

9. Salidas

- **Agregar 200 / Eliminar 200:** { success: true, mensaje }.
- **Listar 200:** [{ productid, fechaAgregado, nombre, brand, price, categoriaid, imagenUrl }].
- **Errores:** Agregar 400 (duplicado) "El producto ya esta en tu wishlist"; 500 en operaciones.
- **UI:** "Agregado a wishlist" / "Eliminado de wishlist"; "Agregado al carrito" al mover.

10. Escenarios (Gherkin)

Escenario 1 — Agregar (CU-10)

Dado un cliente autenticado viendo un producto que no está en su wishlist
 Cuando pulsa el corazón (POST /api/wishlist/agregar)
 Entonces se inserta el ítem, se registra wishlist y el corazón pasa a "lleno"

Escenario 2 — Quitar con toggle (CU-12)

Dado un producto ya en la wishlist (corazón lleno)
 Cuando pulsa de nuevo el corazón (DELETE)
 Entonces se borra físicamente la fila y el corazón vuelve a "vacío" (sin registrar evento)

Escenario 3 — Listar (CU-11)

Dado un cliente con productos guardados
 Cuando solicita GET /api/wishlist/{userId}
 Entonces recibe sus productos con datos del catálogo, orden fecha desc

Escenario 4 — Duplicado (CU-10)

Dado un producto ya presente en la wishlist
 Cuando intenta agregarlo otra vez
 Entonces responde 400 y no crea fila duplicada

11. Criterios de aceptación

ID	Criterio
CA-WIS-001	Agregar un producto nuevo crea fila en wishlist_items y genera evento wishlist.
CA-WIS-002	Agregar un producto presente → 400 sin duplicar.
CA-WIS-003	GET /{userId} devuelve los productos con datos del catálogo (orden fecha desc).
CA-WIS-004	Eliminar borra físicamente la fila y deja de aparecer.
CA-WIS-005	El corazón refleja el estado: agregar lo llena, eliminar lo vacía.
CA-WIS-006	Wishlist sin productos → 200 con lista vacía.
CA-WIS-007	Todas las operaciones rechazan peticiones sin token JWT válido.

12. Restricciones

- Wishlist en retailmind.wishlist_items; eventos en fact_eventos; producto contra productos_catálogo.
- JdbcTemplate (sin JPA); todas las consultas por concatenación (PENDIENTE de seguridad).
- Autenticación obligatoria (no opera de forma anónima).
- Borrado físico (a diferencia del carrito, que usa borrado lógico).
- Stack fijado: Spring Boot 3.5 / Angular 17 standalone con Angular Material.

13. Dependencias

- Módulo de Catálogo (002): resuelve datos del producto y reutiliza registrarEvento; el toggle se muestra en grid/detalle.
- Módulo de Carrito (003): mover de wishlist al carrito invoca POST /api/carrito/agregar.
- Módulo de Autenticación (001): provee el username en sesión.
- ClickHouse con wishlist_items, productos_catálogo y fact_eventos.
- Constitución, Principios II y V.

14. Fuera de alcance

- Catálogo y detalle (002): otro módulo; aquí solo el toggle y el JOIN.
- Carrito y checkout (003): mover al carrito invoca ese módulo.
- Compartir la wishlist o múltiples listas por usuario: no implementado.
- Notificaciones de cambio de precio/stock: no implementadas.
- Paginación de la wishlist: no implementada (devuelve la lista completa).

4.5 Pedidos del cliente (historial)

Feature Branch: 005-operativo-pedidos

1. Nombre de la funcionalidad

Pedidos del cliente (historial)

2. Objetivo

Permitir que un cliente autenticado consulte el historial de sus pedidos y el detalle de cada orden (ítems, cantidades, precios, total, fecha y estado), leyendo las tablas ordenes y orden_items en ClickHouse. Es un módulo de solo lectura que da seguimiento a las compras y refuerza la confianza y el retorno del cliente.

3. Usuarios / actores

- **CLIENTE autenticado:** consulta su historial y el detalle de cada orden. Actor principal.
- **ADMIN autenticado:** dispone de /api/pedidos/admin/todos (módulo administrativo, fuera de alcance).
- **Sistema (backend):** recupera las órdenes filtradas por usuario y las enriquece con sus ítems.

Requiere autenticación: /api/pedidos/{userId} bajo authenticated() y /mis-pedidos protegida por authGuard. /api/pedidos/admin/** exige rol ADMIN.

4. Contexto del problema

Tras comprar (checkout del módulo 003), el cliente necesita revisar qué compró, cuándo y por cuánto. Sin un historial accesible se pierde transparencia y baja la recompra. RetailMind expone al cliente sus propias órdenes con su detalle, ordenadas (lo más reciente primero), manteniendo la separación entre lo que ve un cliente (solo lo suyo) y un administrador (todo).

5. Requisitos funcionales

ID	Requisito
RF-PED-001	Listar pedidos vía GET /api/pedidos/{userId} (requiere autenticación).
RF-PED-002	Filtrar por user_id y ordenar por fecha_orden descendente.
RF-PED-003	Cada orden: ordenId, total, estado, fechaOrden, canal.
RF-PED-004	Cada orden incluye sus ítems (productId, cantidad, precioUnitario, nombre vía JOIN) y numItems.
RF-PED-005	El detalle se entrega embebido en el listado; no hay endpoint de detalle independiente.
RF-PED-006	La vista "Mis pedidos" muestra el historial y permite expandir cada orden.
RF-PED-007	Módulo de solo lectura: no crea, modifica, cancela ni elimina órdenes.
RF-PED-008	Todas las operaciones requieren usuario autenticado.

6. Requisitos no funcionales

ID	Requisito
RNF-PED-001	/api/pedidos/{userId} requiere token JWT válido.
RNF-PED-002	Historial ordenado por fecha_orden descendente.
RNF-PED-003	Acceso a datos con JdbcTemplate sobre ClickHouse (sin JPA).
RNF-PED-004	Ante error, 500 { error }; el cliente degrada a lista vacía sin romper la vista.
RNF-PED-005	PENDIENTE (N+1): una consulta de ítems por cada orden; evaluar consulta agregada.

ID	Requisito
RNF-PED-006	PENDIENTE: no existe umbral de latencia medido para la carga del historial.

7. Reglas de negocio

ID	Regla
RN-PED-001	Un cliente debe ver solo sus órdenes. PENDIENTE: el backend no valida la propiedad (PEND-PED-01).
RN-PED-002	Las órdenes se crean solo en el checkout (003); este módulo no las genera ni modifica.
RN-PED-003	Toda orden nace y permanece COMPLETADA (sin transiciones de estado).
RN-PED-004	En la interfaz, opera sobre el username en sesión.
RN-PED-005	El total mostrado es el persistido al checkout (no se recalcula).

8. Entradas

- **Listar (GET /api/pedidos/{userId}):** userId en la ruta.
- **Autenticación:** Authorization: Bearer <token>.
- **(Fuera de alcance) Todos los pedidos (GET /api/pedidos/admin/todos):** solo ADMIN.

9. Salidas

- **Listar 200:** [{ orderId, total, estado, fechaOrden, canal, numItems, items:[{ productid, cantidad, precioUnitario, nombre }] }].
- **Sin pedidos:** arreglo vacío [].
- **Errores:** Listar 500 { error }; en UI la vista muestra historial vacío (degradación silenciosa).

10. Escenarios (Gherkin)

Escenario 1 — Ver mis pedidos (CU-13)

Dado un cliente autenticado con órdenes
 Cuando solicita GET /api/pedidos/{userId} con su username
 Entonces recibe 200 con sus órdenes (fecha desc) y los campos básicos

Escenario 2 — Ver detalle (CU-14)

Dado un cliente viendo su historial
 Cuando expande una orden
 Entonces ve sus ítems embebidos (de la misma respuesta del listado)

Escenario 3 — Cliente sin pedidos (CU-13)

Dado un cliente que nunca compró
 Cuando solicita su historial
 Entonces recibe 200 con lista vacía y la vista lo refleja

Escenario 4 — Orden ajena (comportamiento actual)

Dado un cliente autenticado
 Cuando solicita GET /api/pedidos/{otroUsername}
 Entonces el backend devuelve las órdenes de ese otro usuario (no valida propiedad — PENDIENTE)

11. Criterios de aceptación

ID	Criterio
CA-PED-001	GET /{userId} devuelve las órdenes del usuario (fecha desc) con sus campos básicos.
CA-PED-002	Cada orden incluye sus ítems (productid, cantidad, precioUnitario, nombre) y numItems.

ID	Criterio
CA-PED-003	El detalle de ítems viene de la misma respuesta del listado (sin endpoint separado).
CA-PED-004	Un cliente sin órdenes recibe 200 con lista vacía.
CA-PED-005	El módulo no expone operaciones de escritura (solo lectura).
CA-PED-006	La operación rechaza peticiones sin token JWT válido.
CA-PED-007	PENDIENTE: impedir que un cliente obtenga órdenes de otro (hoy no se impide en la API).

12. Restricciones

- Órdenes en retailmind.ordenes y orden_items; nombre del producto contra productos_catalogo.
- JdbcTemplate (sin JPA); consultas por concatenación (PENDIENTE de seguridad).
- Solo lectura: el módulo no crea ni altera órdenes.
- Autenticación obligatoria; estado único: toda orden está COMPLETADA.
- Stack fijado: Spring Boot 3.5 / Angular 17 standalone con Angular Material.

13. Dependencias

- Módulo de Carrito (003): genera las órdenes que este módulo consulta.
- Módulo de Catálogo (002): provee el nombre del producto vía JOIN.
- Módulo de Autenticación (001): provee el username en sesión.
- Módulo administrativo de pedidos (00-15): /api/pedidos/admin/todos pertenece a ese módulo.
- ClickHouse con ordenes, orden_items y productos_catalogo; Constitución, Principios II y V.

14. Fuera de alcance

- Creación de la orden (checkout): pertenece al módulo de Carrito (003).
- Supervisión administrativa de todos los pedidos (00-15): módulo administrativo aparte.
- Cancelación, devolución o cambio de estado de un pedido: no implementado.
- Facturación / comprobantes: no implementada en este módulo.
- Seguimiento logístico / tracking: no implementado.
- Paginación del historial: no implementada.

4.6 Perfil del cliente

Feature Branch: 006-operativo-perfil

1. Nombre de la funcionalidad

Perfil del cliente

2. Objetivo

Permitir que un cliente autenticado consulte su propio perfil —identidad y estadísticas derivadas de su actividad— y edite sus datos editables (email y contraseña), operando sobre usuarios_sistema y agregando métricas desde ordenes, wishlist_items y fact_eventos. Refuerza la gestión segura de la identidad del usuario.

3. Usuarios / actores

- **CLIENTE autenticado:** consulta su perfil y estadísticas, actualiza email y cambia contraseña. Actor principal.
- **ADMIN autenticado:** consulta/edita su propio perfil como cualquier usuario (la administración de terceros es otro módulo).
- **Sistema (backend):** recupera la identidad, calcula estadísticas y aplica las actualizaciones.

Requiere autenticación: rutas /api/perfil/** bajo authenticated() y /perfil protegida por authGuard. Opera sobre el username en sesión.

4. Contexto del problema

El cliente necesita ver quién es en la plataforma, entender su actividad y mantener actualizados sus datos de acceso (email y contraseña). Sin perfil, la gestión de identidad queda dispersa y el cliente no puede corregir sus datos de forma autónoma. RetailMind expone un perfil personal seguro, donde cada cliente ve y edita únicamente lo suyo (alineado con OT-07 y OE-04).

5. Requisitos funcionales

ID	Requisito
RF-PER-001	Exponer el perfil vía GET /api/perfil/{username}: username, email, rol, activo, fechaCreacion.
RF-PER-002	Incluir 6 estadísticas: totalCompras, totalGastado, productosWishlist, totalEventos, categoriaFavorita y canalPreferido.
RF-PER-003	El email corresponde al campo nombre de usuarios_sistema (no hay columna email dedicada).
RF-PER-004	Actualizar email vía PUT /api/perfil/{username}/email (email no vacío).
RF-PER-005	Cambiar contraseña vía PUT /api/perfil/{username}/password (actual correcta, nueva distinta, hash BCrypt).
RF-PER-006	Devolver 404 si el usuario no existe.
RF-PER-007	El cliente valida en formulario el email y la contraseña nueva (≥ 6 , confirmación, distinta de la actual).
RF-PER-008	Todas las operaciones requieren usuario autenticado.

6. Requisitos no funcionales

ID	Requisito
RNF-PER-001	Rutas /api/perfil/** requieren token JWT válido.
RNF-PER-002	Contraseñas con BCrypt; nunca en texto plano ni devueltas en el perfil.
RNF-PER-003	Actualizaciones de email y contraseña síncronas (mutations_sync = 1).
RNF-PER-004	Las 6 estadísticas se calculan desde usuarios_sistema, ordenes, wishlist_items y fact_eventos.

ID	Requisito
RNF-PER-005	Cálculo tolerante a fallos: ante error, 0 o "Sin datos" sin romper el perfil.
RNF-PER-006	La contraseña nueva en cliente: mínimo 6 caracteres y confirmación coincidente.
RNF-PER-007	PENDIENTE: el cliente mide queryMs pero no hay umbral definido.

7. Reglas de negocio

ID	Regla
RN-PER-001	Un cliente debe ver y editar solo su perfil. PENDIENTE: el backend no valida la propiedad (PEND-PER-01).
RN-PER-002	La nueva contraseña debe ser distinta de la actual.
RN-PER-003	El cambio exige que la passwordActual coincida (BCrypt).
RN-PER-004	El email se persiste en el campo nombre de usuarios_sistema.
RN-PER-005	Sin actividad, las estadísticas muestran 0 o "Sin datos".
RN-PER-006	En la interfaz, opera sobre el username en sesión.
RN-PER-007	No se permite editar username ni rol desde el perfil.

8. Entradas

- **Ver perfil (GET /api/perfil/{username}):** username en la ruta.
- **Actualizar email (PUT /{username}/email):** { email }.
- **Cambiar contraseña (PUT /{username}/password):** { passwordActual, passwordNuevo }.
- **Autenticación:** Authorization: Bearer <token>.

9. Salidas

- **Ver perfil 200:** { username, email, rol, activo, fechaCreacion, totalCompras, totalGastado, productosWishlist, totalEventos, categoriaFavorita, canalPreferido }.
- **Actualizar 200:** { success: true, mensaje }.
- **Errores:** 404 (usuario no existe); 400 (email requerido / faltan campos / actual incorrecta / nueva igual a la actual); 500.

10. Escenarios (Gherkin)

Escenario 1 — Ver perfil con estadísticas (CU-008)

Dado un cliente autenticado
 Cuando solicita GET /api/perfil/{username} con su username
 Entonces recibe 200 con su identidad y las 6 estadísticas (0 o "Sin datos" si faltan)

Escenario 2 — Editar email (CU-008)

Dado un email válido en el formulario
 Cuando envía PUT /{username}/email con email no vacío
 Entonces actualiza el campo nombre y responde "Email actualizado correctamente"

Escenario 3 — Datos inválidos (CU-008)

Cuando envía email vacío → 400 "El email es requerido"
 Cuando la contraseña actual es incorrecta → 400 "La contraseña actual es incorrecta"
 Cuando la nueva es igual a la actual → 400 "La nueva contraseña debe ser diferente a la actual"

Escenario 4 — Perfil ajeno (comportamiento actual)

Dado un cliente autenticado

Cuando solicita GET/PUT /api/perfil/{otroUsername}
Entonces el backend opera sobre ese otro usuario (no valida propiedad — PENDIENTE)

11. Criterios de aceptación

ID	Criterio
CA-PER-001	GET /{username} devuelve la identidad y las 6 estadísticas.
CA-PER-002	Las estadísticas se calculan de ordenes, wishlist_items y fact_eventos; sin datos, 0/"Sin datos".
CA-PER-003	Actualizar email con valor no vacío persiste en usuarios_sistema.nombre.
CA-PER-004	Email vacío → 400 "El email es requerido".
CA-PER-005	Cambiar contraseña exige la actual correcta y una nueva distinta; guarda hash BCrypt.
CA-PER-006	Actual incorrecta o nueva igual a la actual → 400 con su mensaje.
CA-PER-007	Usuario inexistente → 404.
CA-PER-008	Todas las operaciones rechazan peticiones sin token JWT válido.
CA-PER-009	PENDIENTE: impedir ver/editar el perfil de otro usuario (hoy no se impide en la API).

12. Restricciones

- Identidad en retailmind.usuarios_sistema; estadísticas desde ordenes, wishlist_items, fact_eventos, dim_producto y dim_categoria.
- JdbcTemplate (sin JPA); consultas y mutaciones por concatenación (PENDIENTE de seguridad).
- Campos editables: solo email (campo nombre) y contraseña; no username ni rol.
- Autenticación obligatoria.
- Stack fijado: Spring Boot 3.5 / Angular 17 standalone con Angular Material.

13. Dependencias

- Módulo de Autenticación (001): comparte usuarios_sistema, repositorio y PasswordEncoder (BCrypt).
- Módulos de Carrito/Pedidos (003/005): ordenes alimenta totalCompras y totalGastado.
- Módulo de Wishlist (004): wishlist_items alimenta productosWishlist.
- Módulo de Catálogo (002): fact_eventos + dim_producto/dim_categoria alimentan categoriaFavorita y canalPreferido.
- ClickHouse con las tablas mencionadas; Constitución, Principio V.

14. Fuera de alcance

- Autenticación / login (001): este módulo solo consume la sesión establecida.
- Administración de usuarios por ADMIN (00-13): módulo administrativo aparte.
- Recuperación / restablecimiento de contraseña: no implementada.
- Edición de username o rol desde el perfil: no permitida.
- Avatar / foto de perfil: no implementado.
- Historial de cambios del perfil / auditoría: no implementado.

4.7 Recomendaciones

Feature Branch: 007-operativo-recomendaciones

1. Nombre de la funcionalidad

Recomendaciones

2. Objetivo

Aumentar la conversión y el engagement ofreciendo al cliente autenticado productos relevantes: un motor calcula sus categorías de mayor interés mediante un scoring ponderado por tipo de evento y recomienda productos no interactuados; ofrece similares a uno en pantalla (misma categoría, precio $\pm 30\%$); y, ante poca actividad, recurre a productos populares (fallback).

3. Usuarios / actores

- **CLIENTE autenticado:** recibe recomendaciones personalizadas, ve similares en el detalle y, con poca actividad, ve populares.
- **Motor de recomendaciones:** componente lógico que calcula el scoring, selecciona no interactuados, computa similares y aplica el fallback.
- **ADMIN autenticado:** puede consumir el módulo como cualquier usuario autenticado.

Requiere autenticación: rutas `/api/recomendaciones/**` bajo `authenticated()` y vistas `(/recomendaciones, /shop/producto/:id)` protegidas por `authGuard`. Opera sobre el `username` en sesión.

4. Contexto del problema

Un catálogo de ~1.200 productos abruma al cliente y dificulta el descubrimiento, reduciendo la conversión. RetailMind guía al cliente hacia productos relevantes a partir de su comportamiento (compras, carrito, wishlist, clics, vistas) y del producto que mira, manteniendo la frescura (no recomendar lo ya interactuado) y cubriendo el arranque en frío mediante populares. Busca maximizar conversiones (OE-01) y reforzar el engagement (OT-02) con personalización (OE-04).

5. Requisitos funcionales

ID	Requisito
RF-REC-001	Exponer recomendaciones vía GET <code>/api/recomendaciones/{username}</code> (requiere autenticación).
RF-REC-002	Contar eventos del usuario y decidir personalizado (≥ 10) o fallback (< 10).
RF-REC-003	En personalizado, calcular las top 3 categorías por scoring ponderado (RN-REC-001).
RF-REC-004	Recomendar productos activos de las top categorías, excluyendo los ya interactuados, orden aleatorio, hasta 12.
RF-REC-005	Si quedan menos de 4, completar con productos populares.
RF-REC-006	En fallback (< 10), devolver populares globales hasta 12, con <code>esPersonalizado = false</code> y mensaje de fallback.
RF-REC-007	Exponer similares vía GET <code>/api/recomendaciones/{username}/similares/{productid}</code> .
RF-REC-008	Similares: misma categoría, precio $\pm 30\%$, activos, excluyendo el propio y los interactuados, orden aleatorio, hasta 6.
RF-REC-009	Respuesta: <code>username</code> , <code>totalEventos</code> , <code>recomendaciones</code> , <code>esPersonalizado</code> , <code>tipo</code> , <code>mensajeFallback</code> , <code>categoriaFavorita</code> , <code>queryMs</code> .
RF-REC-010	Degradación segura: sin categorías o ante fallo recurre a populares; <code>fact_eventos</code> vacía $\rightarrow$ catálogo por stock; error general $\rightarrow$ 500.

ID	Requisito
RF-REC-011	Todas las operaciones requieren usuario autenticado.

6. Requisitos no funcionales

ID	Requisito
RNF-REC-001	Rutas /api/recomendaciones/** requieren token JWT válido.
RNF-REC-002	Umbral de personalización: 10 eventos ($< 10 \rightarrow$ fallback; $\geq 10 \rightarrow$ personalizado).
RNF-REC-003	Pesos del scoring: purchase = 5, add_to_cart = 3, wishlist = 2, click = 1, otras = 0.5 (incluye view).
RNF-REC-004	Rango de precio de similares: $\pm 30\%$ ($\text{price} \times 0.7$ a $\text{price} \times 1.3$).
RNF-REC-005	Límites: 12 (personalizado/populares), 6 (similares), 3 (top categorías); relleno con populares si < 4 .
RNF-REC-006	Incluir el tiempo de cómputo (queryMs) en la respuesta.
RNF-REC-007	Cálculo tolerante a fallos en cada paso sin romper la respuesta.
RNF-REC-008	Acceso a datos con JdbcTemplate sobre ClickHouse (sin JPA).
RNF-REC-009	PENDIENTE: se reporta queryMs pero no hay umbral de latencia definido.

7. Reglas de negocio

ID	Regla
RN-REC-001	Score por categoría: purchase = 5, add_to_cart = 3, wishlist = 2, click = 1, otra acción = 0.5. Las 3 mayores son las top categorías.
RN-REC-002	Menos de 10 eventos $\rightarrow$ fallback de populares (no personalizado).
RN-REC-003	Se excluyen los productos con cualquier evento previo del usuario.
RN-REC-004	Los similares son de la misma categoría y precio entre 70% y 130% del base.
RN-REC-005	Solo se recomiendan productos activos (activo = 1).
RN-REC-006	Personalizadas y similares se ordenan aleatoriamente (rand()): no determinista.
RN-REC-007	Los populares se determinan por el número de compras (purchase) global.
RN-REC-008	En la interfaz, opera sobre el username en sesión.

8. Entradas

- **Recomendaciones** (GET /api/recomendaciones/{username}): username en la ruta.
- **Similares** (GET /{username}/similares/{productid}): username y productid en la ruta.
- **Autenticación**: Authorization: Bearer <token>.

9. Salidas

- **200 (personalizado)**: { ..., recomendaciones:[...], esPersonalizado: true, tipo: "personalizado", categoriaFavorita, queryMs }.
- **200 (fallback)**: { ..., esPersonalizado: false, tipo: "populares", mensajeFallback, categoriaFavorita: null }.
- **Similares 200**: hasta 6 productos; lista vacía si el base no existe/está inactivo o no hay similares.
- **Errores**: Recomendaciones/Similares 500 { error } (el servicio suele devolver lista vacía antes de propagar).

10. Escenarios (Gherkin)

Escenario 1 — Personalizadas por top categorías (CU-009)

Dado un cliente con ≥ 10 eventos
 Cuando solicita GET /api/recomendaciones/{username}
 Entonces recibe hasta 12 productos de sus top 3 categorías, sin los ya interactuados

Escenario 2 — Similares (CU-009)

Dado un producto en pantalla
 Cuando solicita similares
 Entonces recibe hasta 6 de la misma categoría con precio $\pm 30\%$, excluyendo el propio y los vistos

Escenario 3 — Fallback por poca actividad (CU-009)

Dado un usuario con < 10 eventos
 Cuando solicita recomendaciones
 Entonces recibe productos populares con esPersonalizado = false y mensaje de fallback

Escenario 4 — Sin historial (CU-009)

Dado un usuario sin eventos
 Cuando solicita recomendaciones
 Entonces el motor degrada a populares / catálogo por stock sin romper la respuesta

11. Criterios de aceptación

ID	Criterio
CA-REC-001	≥ 10 eventos $\rightarrow$ recomendaciones personalizadas de las top 3 categorías.
CA-REC-002	Los recomendados excluyen productos ya interactuados por el usuario.
CA-REC-003	Similares: misma categoría, precio $\pm 30\%$, activos, hasta 6.
CA-REC-004	< 10 eventos $\rightarrow$ fallback de populares con su mensaje.
CA-REC-005	La respuesta incluye esPersonalizado, tipo, categoriaFavorita y queryMs.
CA-REC-006	Ante fallo de una consulta, el motor degrada sin romper la respuesta.
CA-REC-007	Todas las operaciones rechazan peticiones sin token JWT válido.

12. Restricciones

- Fuentes: fact_eventos, dim_producto, productos_catálogo y dim_categoria.
- JdbcTemplate (sin JPA); consultas por concatenación (PENDIENTE de seguridad).
- No determinismo: el orden rand() implica resultados distintos entre llamadas.
- Autenticación obligatoria; solo lectura (el motor no escribe).
- Stack fijado: Spring Boot 3.5 / Angular 17 standalone con Angular Material.

13. Dependencias

- Módulo de Catálogo (002): productos_catálogo provee los recomendables; los similares se muestran en el detalle.
- Módulos de Carrito (003) y Wishlist (004): sus eventos alimentan el scoring.
- Módulo de Autenticación (001): provee el username en sesión.
- ETL / fact_eventos: el historial de eventos es la materia prima del motor.
- ClickHouse con fact_eventos, dim_producto, productos_catálogo y dim_categoria; Constitución, Principios II, IV y V.

14. Fuera de alcance

- Catálogo y detalle (002): aquí solo se consumen como fuente y se inserta el bloque de similares.
- Carrito (003) y Wishlist (004): las acciones de agregar son de esos módulos.

- Registro de impresiones y clics para el CTR: no implementado.
- Modelos de IA/ML entrenados (colaborativo, embeddings): no implementados; es un scoring heurístico.
- Configuración dinámica de pesos/umbrales sin recompilar: no implementada (valores fijos).
- Caché de recomendaciones: no implementada (se calcula en cada petición).

5. Historias de usuario del nivel operativo

Historias de usuario de rol Cliente que constituyen el núcleo de cada módulo operativo (HU-02 a HU-72), extraídas del documento de la Tarea 3 (GA 03). Las historias de rol Administrador y de analítica/eventos corresponden a los niveles táctico/estratégico y quedan fuera de esta especificación.

ID	Historia de usuario	Módulo	CU
HU-02	Como Cliente, quiero iniciar sesión con mi cuenta, para ver mi carrito, wishlist y pedidos personales.	Autenticación	CU-01
HU-04	Como Cliente, quiero recibir un mensaje claro si mis credenciales son incorrectas, para saber que debo verificar mi usuario o contraseña.	Autenticación	CU-01
HU-13	Como Cliente, quiero ver todos los productos disponibles, para encontrar productos que me interesen comprar.	Catálogo	CU-04
HU-14	Como Cliente, quiero ver imagen, nombre, marca y precio de cada producto, para decidir si veo más detalles.	Catálogo	CU-04
HU-16	Como Cliente, quiero navegar entre páginas de productos, para ver todo el catálogo sin lentitud.	Catálogo	CU-04
HU-17	Como Cliente, quiero filtrar productos por categoría, para ver solo los que me interesan.	Catálogo	CU-05
HU-18	Como Cliente, quiero filtrar por rango de precio, para encontrar productos según mi presupuesto.	Catálogo	CU-05
HU-19	Como Cliente, quiero filtrar por marca, para ver solo mis marcas preferidas.	Catálogo	CU-05
HU-20	Como Cliente, quiero limpiar todos los filtros con un click, para volver al catálogo completo.	Catálogo	CU-05
HU-21	Como Cliente, quiero ver la descripción completa de un producto, para entender qué compraré.	Catálogo	CU-06
HU-22	Como Cliente, quiero seleccionar la cantidad a comprar, para agregar varias unidades de una vez.	Catálogo	CU-06
HU-23	Como Cliente, quiero ver la categoría del producto, para identificar rápido el tipo de artículo.	Catálogo	CU-06
HU-25	Como Cliente, quiero agregar productos al carrito con un click, para guardar mis intenciones de compra.	Carrito y checkout	CU-07
HU-26	Como Cliente, quiero ver el número de productos del carrito en tiempo real, para saber cuántos llevo.	Carrito y checkout	CU-07
HU-28	Como Cliente, quiero agregar desde el catálogo y desde el detalle, para tener flexibilidad.	Carrito y checkout	CU-07
HU-29	Como Cliente, quiero ver todos los productos del carrito, para revisar mi selección antes de comprar.	Carrito y checkout	CU-08
HU-30	Como Cliente, quiero eliminar productos del carrito, para ajustar mi selección.	Carrito y checkout	CU-08
HU-31	Como Cliente, quiero ver el total actualizado, para saber cuánto gastaré antes de confirmar.	Carrito y checkout	CU-08
HU-33	Como Cliente, quiero finalizar mi compra con un click, para completar el proceso rápido.	Carrito y checkout	CU-09

ID	Historia de usuario	Módulo	CU
HU-34	Como Cliente, quiero recibir un número de orden al comprar, para dar seguimiento a mi pedido.	Carrito y checkout	CU-09
HU-36	Como Cliente, quiero que el carrito se vacíe automáticamente al comprar, para no limpiarlo manualmente.	Carrito y checkout	CU-09
HU-37	Como Cliente, quiero agregar productos a mi lista de deseos, para guardar artículos sin comprarlos ahora.	Wishlist	CU-10
HU-38	Como Cliente, quiero ver si un producto ya está en mi wishlist, para saberlo de un vistazo.	Wishlist	CU-10
HU-39	Como Cliente, quiero quitar productos con el corazón, para mantener mi wishlist actualizada.	Wishlist	CU-10
HU-41	Como Cliente, quiero ver mis productos guardados en un solo lugar, para acceder rápido a ellos.	Wishlist	CU-11
HU-42	Como Cliente, quiero agregar de la wishlist al carrito directamente, para comprar sin buscar de nuevo.	Wishlist	CU-11
HU-43	Como Cliente, quiero ver un mensaje cuando mi wishlist está vacía, para saber que no he guardado nada.	Wishlist	CU-11
HU-44	Como Cliente, quiero ver el precio actualizado de mis guardados, para saber si cambió.	Wishlist	CU-11
HU-45	Como Cliente, quiero eliminar productos que ya no me interesan, para mantener mi wishlist relevante.	Wishlist	CU-12
HU-46	Como Cliente, quiero que la wishlist se actualice al eliminar, para ver el cambio sin recargar.	Wishlist	CU-12
HU-47	Como Cliente, quiero eliminar desde el catálogo sin entrar a la wishlist, para gestionarla mientras navego.	Wishlist	CU-12
HU-49	Como Cliente, quiero ver todas mis órdenes en un solo lugar, para tener mi historial de compras.	Pedidos	CU-13
HU-50	Como Cliente, quiero ver el total y la fecha de cada orden, para controlar mis gastos.	Pedidos	CU-13
HU-51	Como Cliente, quiero ver un mensaje cuando no tengo pedidos, para saber que no he comprado.	Pedidos	CU-13
HU-52	Como Cliente, quiero ver el estado de cada orden, para saber si está pendiente, completada o cancelada.	Pedidos	CU-13
HU-53	Como Cliente, quiero ver qué productos incluí en cada orden, para recordar qué compré.	Pedidos	CU-14
HU-54	Como Cliente, quiero ver la cantidad y precio de cada producto, para verificar el cobro.	Pedidos	CU-14
HU-55	Como Cliente, quiero expandir y contraer el detalle de cada orden, para ver solo lo que necesito.	Pedidos	CU-14
HU-57	Como Cliente, quiero ver mi información personal, para verificar que mis datos son correctos.	Perfil	CU-15

ID	Historia de usuario	Módulo	CU
HU-58	Como Cliente, quiero ver cuántas compras he hecho, para tener un resumen de mi actividad.	Perfil	CU-15
HU-59	Como Cliente, quiero ver cuántos productos tengo en wishlist, para un resumen de mis guardados.	Perfil	CU-15
HU-60	Como Cliente, quiero ver mi rol en el sistema, para entender mis permisos.	Perfil	CU-15
HU-61	Como Cliente, quiero actualizar mi email, para mantener mi contacto al día.	Perfil	CU-16
HU-62	Como Cliente, quiero cambiar mi contraseña cuando lo necesite, para mantener la seguridad de mi cuenta.	Perfil	CU-16
HU-63	Como Cliente, quiero recibir confirmación al guardar cambios, para saber que se actualizó.	Perfil	CU-16
HU-64	Como Cliente, quiero ver un error si mi contraseña no cumple los requisitos, para crear una válida.	Perfil	CU-16
HU-65	Como Cliente, quiero ver productos recomendados según mi historial, para descubrir artículos relevantes.	Recomendaciones	CU-17
HU-66	Como Cliente, quiero ver productos similares al que veo, para explorar opciones relacionadas.	Recomendaciones	CU-17
HU-68	Como Cliente, quiero agregar recomendados al carrito directamente, para comprar sin pasos extra.	Recomendaciones	CU-17
HU-69	Como Cliente, quiero ver los productos que visité recientemente, para retomar donde quedé.	Recomendaciones	CU-18
HU-70	Como Cliente, quiero ver cuántas veces visité cada producto, para saber cuáles me llaman más.	Recomendaciones	CU-18
HU-72	Como Cliente, quiero limpiar mi historial de navegación, para mantener mi privacidad.	Recomendaciones	CU-18

6. Matriz de trazabilidad consolidada

Vinculación por módulo entre objetivos estratégicos (OE), tácticos (OT) y operativos (OO), casos de uso (CU), historias de usuario (HU) y los requisitos funcionales (RF) principales.

Módulo	OE	OT	OO	CU	HU	RF principales
Autenticación	OE-04	OT-07, OT-08	OO-12, OO-14	CU-01, CU-02	HU-02, HU-04	RF-AUT-001..020
Catálogo	OE-01	OT-01	OO-01	CU-04, CU-05, CU-06	HU-13..23	RF-CAT-001..017
Carrito y checkout	OE-01	OT-01	OO-02	CU-07, CU-08, CU-09	HU-25..36	RF-CAR-001..017
Wishlist	OE-01	OT-02	OO-03	CU-10, CU-11, CU-12	HU-37..47	RF-WIS-001..009
Pedidos	OE-01	OT-02	OO-04	CU-13, CU-14	HU-49..55	RF-PED-001..008
Perfil	OE-04	OT-07	— (sin OO en EVF04)	CU-008 (TA06)	HU-57..64	RF-PER-001..008
Recomendaciones	OE-01, OE-04	OT-02	— (sin OO en EVF04)	CU-009 (TA06)	HU-65..72	RF-REC-001..011

7. Hallazgos y deuda técnica

Consolidado de los puntos PENDIENTE (PEND-*) detectados al especificar el código real de los 7 módulos operativos, agrupados por tema transversal.

ID	Tema	Módulo	Descripción
PEND-PED-01	Seguridad — IDOR	Pedidos	GET /api/pedidos/{userId} no valida la propiedad contra el token: un usuario puede ver pedidos de otro.
PEND-PER-01	Seguridad — IDOR	Perfil	GET/PUT /api/perfil/{username} no valida propiedad; permite ver/editar perfil ajeno.
PEND-REC-04	Seguridad — IDOR	Recomendaciones	GET /api/recomendaciones/{username} no valida propiedad; expone indirectamente el historial de otro.
PEND-AUT-04	Seguridad — SQL concatenado	Autenticación	save() construye el INSERT por concatenación con escape manual (no parametrizado).
PEND-CAT-03	Seguridad — SQL concatenado	Catálogo	Las consultas de catálogo (listado, detalle, evento) por concatenación de cadenas.
PEND-CAR-03	Seguridad — SQL concatenado	Carrito	Inserciones y mutaciones (agregar, eliminar, checkout) por concatenación.
PEND-WIS-02	Seguridad — SQL concatenado	Wishlist	count, insert, select y delete por concatenación de cadenas.
PEND-PED-02	Seguridad — SQL concatenado	Pedidos	Consultas del historial por concatenación de cadenas.
PEND-PER-02	Seguridad — SQL concatenado	Perfil	Consultas y mutaciones del perfil por concatenación de cadenas.
PEND-REC-05	Seguridad — SQL concatenado	Recomendaciones	Consultas del motor por concatenación de cadenas.
PEND-AUT-01	KPI no instrumentado	Autenticación	No se contabilizan intentos de login (KPI de accesos no autorizados no calculable).
PEND-CAT-04	KPI no instrumentado	Catálogo	CTR de catálogo no calculable: solo evento view del detalle; faltan impresiones/clics.
PEND-CAR-06	KPI no instrumentado	Carrito	Tiempo de checkout derivable de add_to_cart→purchase, pero sin cálculo/endpoint.
PEND-WIS-05	KPI no instrumentado	Wishlist	Wishlist rate derivable de eventos wishlist/view, pero sin cálculo/endpoint.
PEND-PED-05	KPI no instrumentado	Pedidos	Frecuencia de compra derivable de ordenes, pero sin cálculo/endpoint.
PEND-PER-05	KPI no instrumentado	Perfil	El perfil no expone un KPI claro en EVF04 ni en el código.
PEND-REC-01	KPI no instrumentado	Recomendaciones	CTR de recomendaciones (meta 15%): no se registran impresiones ni clics.
PEND-CAR-02	Integridad de datos	Carrito	El checkout no es atómico (ClickHouse sin transacciones): un fallo deja estado parcial.
PEND-CAR-04	Integridad de datos	Carrito	No se valida ni descuenta stock al agregar ni en checkout.

ID	Tema	Módulo	Descripción
PEND-CAR-05	Integridad de datos	Carrito	No hay pasarela de pago ni estados de orden; toda orden nace COMPLETADA.
PEND-PED-06	Integridad de datos	Pedidos	No existe máquina de estados del pedido (siempre COMPLETADA).
PEND-WIS-04	Integridad de datos	Wishlist	La wishlist usa borrado físico (sin historial de eliminaciones).
PEND-REC-03	Integridad de datos	Recomendaciones	El motor mezcla dim_producto y productos_catalogo; si los IDs no coinciden, degrada las recomendaciones.
PEND-AUT-03	Validación débil	Autenticación	register no valida formato de email ni fuerza de contraseña; el email se guarda en el campo nombre.
PEND-PER-03	Validación débil	Perfil	El backend no valida el formato del email (solo que no esté vacío).
PEND-PER-04	Validación débil	Perfil	El backend no valida longitud/fuerza de la contraseña nueva (mínimo 6 solo en cliente).
PEND-CAT-01	Desfase UI/API	Catálogo	Filtros de marca/precio soportados por la API pero no cableados en la UI del grid.
PEND-CAT-02	Desfase UI/API	Catálogo	Campo de búsqueda por texto en la UI que no filtra (backend sin búsqueda por texto).
PEND-CAR-01	Desfase UI/API	Carrito	No existe 'modificar cantidad'; cada agregar inserta una fila nueva (posibles duplicados).
PEND-WIS-01	Comportamiento	Wishlist	La eliminación no registra evento; solo el alta registra wishlist.
PEND-WIS-03	Comportamiento	Wishlist	Al agregar a wishlist no se valida que el producto exista.
PEND-REC-02	Comportamiento	Recomendaciones	El peso 0.5 es el valor por defecto del multilf (cualquier acción no listada, incluida view).
PEND-REC-06	Comportamiento	Recomendaciones	'Productos vistos' excluye todos los productos con cualquier evento, no solo view.
PEND-REC-07	Comportamiento	Recomendaciones	Recomendaciones no deterministas (orden rand()).
PEND-REC-08	Configurabilidad	Recomendaciones	Pesos, umbral (10) y rango (±30%) fijos en código, no configurables.
PEND-AUT-05	Otros	Autenticación	El logout no invalida el token en el servidor (solo limpia el cliente).
PEND-AUT-06	Otros	Autenticación	El rol VIEWER está definido en el enum pero no se usa.
PEND-PED-03	Rendimiento	Pedidos	Patrón N+1 al cargar los ítems del historial (una consulta por orden).